📊 Kadane's Algorithm Master Guide

Master Dynamic Programming for Subarray Problems with Visual Explanations

📋 Table of Contents

1. What is Kadane's Algorithm?
2. The Core Technique (Thought Process)
3. When to Use Kadane's Algorithm
4. Kadane vs Sliding Window
5. Problem 1: Maximum Subarray Sum
6. Problem 2: Minimum Subarray Sum
7. Problem 3: Maximum Product Subarray
8. Problem 4: Max Sum with One Deletion
9. Problem 5: Maximum Absolute Sum
10. Problem 6: Circular Subarray Sum
11. Quick Reference Summary

1. What is Kadane's Algorithm? 🤔

Simple Definition:

Kadane's Algorithm is a **Dynamic Programming pattern** for finding the "best" (maximum or minimum) value in a **contiguous subarray**. It works by calculating the **"Best Ending at Index i"** and making a local decision at each step.

Key Characteristics:

- ✓ **Works with negative numbers** (unlike sliding window)
- ✓ **Contiguous subarrays only**
- ✓ **O(n) time, O(1) space** - very efficient!
- ✓ **Decision-based approach** at each index
- ✓ **Can be adapted** for sum, product, frequency

⚠️ **Important:** Kadane's is NOT about memorizing code - it's about understanding the thought process and decision-making at each index!

2. The Core Technique: "The Party Choice" 🎉

The Fundamental Question at Each Index:

At every index i, ask yourself:

"Should I join the previous party, or start my own party?"

Visual Representation:

```
    [ Previous History ]      [ Current Element (You) ]
           (i-1)                        (i)
             |                           |
             v                           v
   ─────────────────────────────────────────────
   |   THE DECISION FORK (Best Ending at i)         |
   ─────────────────────────────────────────────
            /                              \
           /                                \
          /                                  \
  OPTION A: JOIN THE PARTY           OPTION B: START NEW PARTY
  ════════════════════════           ════════════════════════
  (Previous Best + Current)          (Current Element Alone)

  "The history is positive/          "The history is negative/
   helpful. I will extend             burden. I will start a
   the streak."                       fresh streak."

   bestEnding + arr[i]                arr[i]
```

The Two Choices in Detail:

Choice 1 Join the Previous Party

When: The previous "bestEnding" is helpful (positive for max, negative for min)
Action: `bestEnding = bestEnding + arr[i]`
Meaning: Extend the existing subarray by including current element
Example: If bestEnding = 5 and arr[i] = 3, join to get 8

Choice 2 Start Your Own Party

When: The previous "bestEnding" is a burden
Action: `bestEnding = arr[i]`
Meaning: Discard all history and start fresh from current element
Example: If bestEnding = -10 and arr[i] = 5, start new at 5

The Update Step:

After making the local decision at index i, compare the new `bestEnding` with the global `result` to see if you've found a new overall maximum/minimum.

```
// After calculating bestEnding at index i
result = Math.max(result, bestEnding);  // For maximum problems
result = Math.min(result, bestEnding);  // For minimum problems
```

Visual State Machine:

```
😊 VISUAL STATE MACHINE: How Kadane's Thinks

┌──────────────────────────────────────┐
│              START OF ARRAY          │
│            Initialize: i = 0         │
└──────────────────────────────────────┘
                    ↓
┌──────────────────────────────────────┐
│               AT INDEX i             │
│  Current State:                      │
│  • arr[i] = current element          │
│  • bestEnding = best sum ending at (i-1) │
│  • result = global maximum so far    │
└──────────────────────────────────────┘
                    ↓
            ╔════════════════╗
            ║ DECISION TIME! 🤔 ║
            ╚════════════════╝
                    ↓
          ┌─────────┴─────────┐
          ↓                   ↓
┌────────────┐        ┌────────────┐
│ OPTION A   │        │ OPTION B   │
│ Join Party │        │ Start New  │
└────────────┘        └────────────┘
          ↓                   ↓
  bestEnding + arr[i]      arr[i]
          ↓                   ↓
     Is history          Forget history
     helpful? ✅          Start fresh 🆕
          ↓                   ↓
          └─────────┬─────────┘
                    ↓
            ┌────────────────┐
            │ Math.max(A, B) │
            │ Choose WINNER! │
            └────────────────┘
                    ↓
┌──────────────────────────────────────┐
│              UPDATE STATE            │
│  bestEnding = winner                 │
│  result = Math.max(result, bestEnding) │
└──────────────────────────────────────┘
                    ↓
             i++ (move right)
                    ↓
            ┌────────────────┐
            │ More elements? │
            └────────────────┘
               ↓YES      ↓NO
          [Loop Back]  [RETURN result]
```

3. When to Use Kadane's Algorithm 🎯

Recognition Checklist - ALL Must Be True:

Criterion	What to Look For	Example
1. Data Structure	Input is an **Array**	int[] nums = {-2, 1, -3, 4}
2. Content Type	Contains **positives, negatives, zeros**	[-1, 5, -3, 0, 2]
3. Target	Find a **Contiguous Subarray**	"Find the subarray..."
4. Objective	Find the **"Best"** value (Max/Min)	"Maximum sum", "Minimum product"
5. Operation	Involves **Sum, Product, or Frequency**	Sum of elements, product of elements

Common Keywords That Indicate Kadane's:

- ✓ "Maximum subarray sum"
- ✓ "Minimum subarray sum"
- ✓ "Maximum product subarray"
- ✓ "Largest contiguous sum"
- ✓ "Best subarray" (with sum/product)
- ✓ "Contiguous elements" + "maximum/minimum"

Variations to Watch For:

Advanced Variations:

- 🔶 One deletion allowed
- 🔶 Circular array (last connects to first)
- 🔶 Absolute sum (|sum|)
- 🔶 K deletions allowed
- 🔶 With constraints on length

4. Kadane vs Sliding Window 🆚

⚠️ Why Sliding Window FAILS with Negatives

Sliding Window loses its **decision-making power** when negative numbers are present!

The Problem with Sliding Window:

```
With Positive Numbers ONLY:
──────────────────────────
Expand window (high++)  → Sum INCREASES  ✅ Predictable
Shrink window (low++)   → Sum DECREASES  ✅ Predictable

With Negative Numbers:
──────────────────────────
Expand window (high++) → Might add -5  → Sum DECREASES  ❌ Unpredictable!
Shrink window (low++)  → Might remove -5 → Sum INCREASES  ❌ Unpredictable!

Result: You don't know which pointer to move! 🤯
```

Comparison Table:

Feature	Sliding Window	Kadane's Algorithm
Input Type	Works with positive numbers	Handles positives, negatives, zeros
Logic	Adjust start/end pointers (low/high)	Calculate "Best Ending at i"
Decision	"Should I grow or shrink?"	"Should I join or start new?"
Primary Use	Subarrays with specific constraints (sum=k)	Best (Max/Min) sum or product
Time Complexity	O(n)	O(n)

Visual Demonstration of the Failure:

```
❌ WHY SLIDING WINDOW FAILS WITH NEGATIVES:

Scenario 1: All Positives (Sliding Window Works ✅)
────────────────────────────────────────
Array: [1, 2, 3, 4, 5]
        ├─────┤
       low   high    Sum = 1+2+3 = 6

Expand (high++):  Add 4 → Sum = 10 ↑ INCREASES (Predictable!)
Shrink (low++):   Remove 1 → Sum = 9 ↓ DECREASES (Predictable!)

Decision: CLEAR! Move pointers based on what you need.

Scenario 2: With Negatives (Sliding Window Breaks ❌)
────────────────────────────────────────
Array: [5, -3, 2, -1, 4]
        ├────────┤
       low      high    Sum = 5+(-3)+2 = 4

Expand (high++):  Add -1 → Sum = 3 ↓ DECREASED! (Unexpected!)
                  What if next is +10? Should we expand?

Shrink (low++):   Remove 5 → Sum = -1 ↓ DECREASED! (Unexpected!)
                  We removed positive but sum went down!

Decision: CONFUSED! 😵 Can't predict which pointer to move!

The Problem:
┌──────────────────────────────────────┐
│ Negatives make it IMPOSSIBLE to know if: │
│ • Expanding will increase or decrease sum │
│ • Shrinking will increase or decrease sum │
│ → Lost decision-making power! 🤯          │
└──────────────────────────────────────┘
```

5. Problem 1: Maximum Subarray Sum 📈

Problem Statement:

LeetCode 53 - Maximum Subarray

Given an integer array `nums`, find the contiguous subarray (containing at least one number) which has the largest sum and return its sum.

Example:

```
Input: nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
Explanation: [4, -1, 2, 1] has the largest sum = 6
```

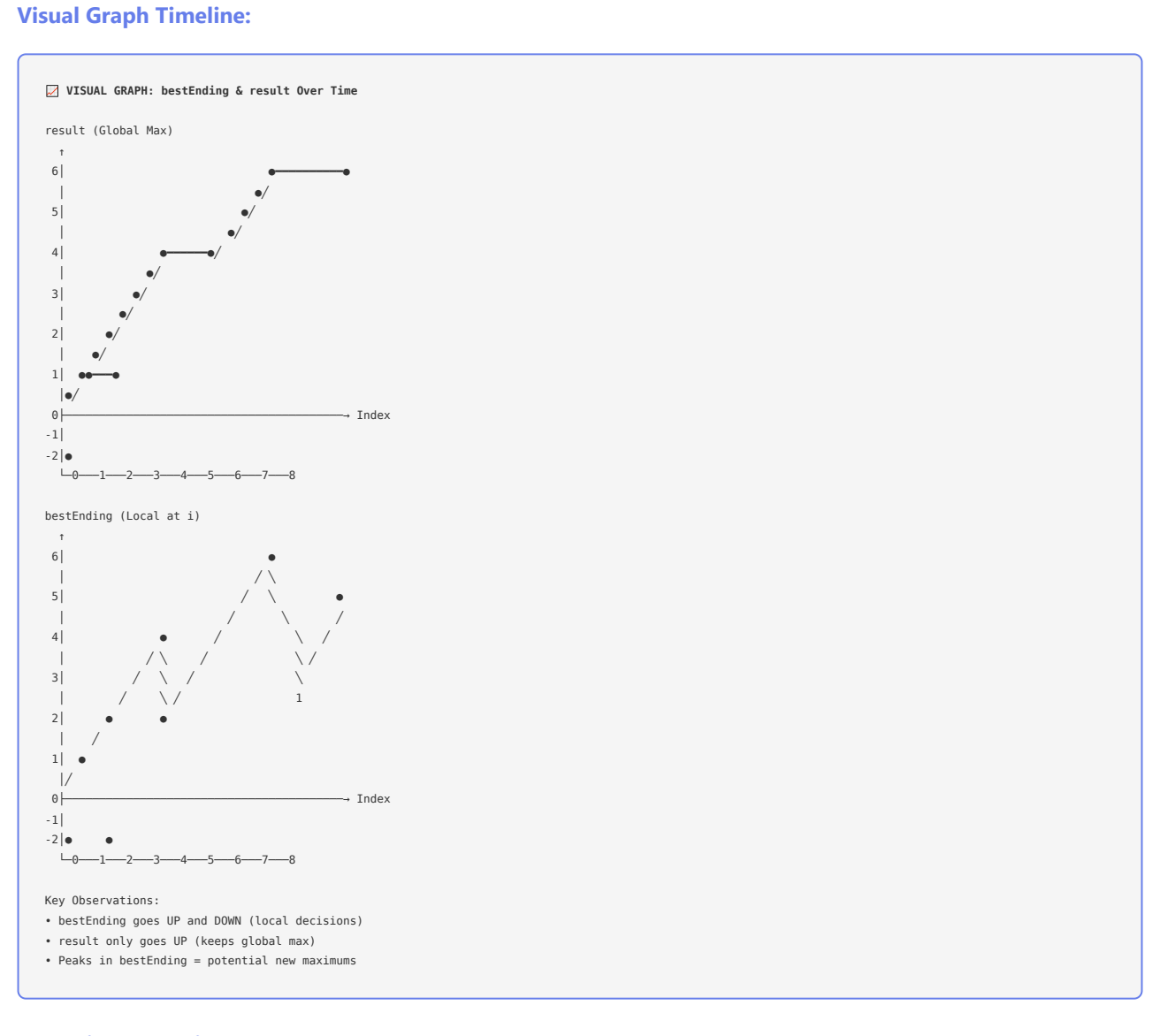
Visual Walkthrough:

```
🔍 DETAILED TRACE: Maximum Subarray Sum

Array: [-2,  1, -3,  4, -1,  2,  1, -5,  4]

┌──────────────────────────────────────┐
│ INDEX 0: arr[0] = -2                 │
├──────────────────────────────────────┤
│  No history (first element)          │
│  bestEnding = -2                     │
│  result = -2                         │
│  Subarray: [-2]                      │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 1: arr[1] = 1                  │
├──────────────────────────────────────┤
│  History: -2 (BURDEN! ❌)             │
│  Choice A: Join → -2 + 1 = -1        │
│  Choice B: New  → 1         ✅ WINNER │
│  bestEnding = 1                      │
│  result = max(-2, 1) = 1             │
│  Subarray: [1]                       │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 2: arr[2] = -3                 │
├──────────────────────────────────────┤
│  History: 1 (HELPFUL! ✅)             │
│  Choice A: Join → 1 + (-3) = -2  ✅ WINNER │
│  Choice B: New  → -3                 │
│  bestEnding = -2                     │
│  result = max(1, -2) = 1             │
│  Subarray: [1, -3]                   │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 3: arr[3] = 4                  │
├──────────────────────────────────────┤
│  History: -2 (BURDEN! ❌)             │
│  Choice A: Join → -2 + 4 = 2         │
│  Choice B: New  → 4         ✅ WINNER │
│  bestEnding = 4                      │
│  result = max(1, 4) = 4              │
│  Subarray: [4]                       │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 4: arr[4] = -1                 │
├──────────────────────────────────────┤
│  History: 4 (HELPFUL! ✅)             │
│  Choice A: Join → 4 + (-1) = 3  ✅ WINNER │
│  Choice B: New  → -1                 │
│  bestEnding = 3                      │
│  result = max(4, 3) = 4              │
│  Subarray: [4, -1]                   │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 5: arr[5] = 2                  │
├──────────────────────────────────────┤
│  History: 3 (HELPFUL! ✅)             │
│  Choice A: Join → 3 + 2 = 5   ✅ WINNER │
│  Choice B: New  → 2                  │
│  bestEnding = 5                      │
│  result = max(4, 5) = 5              │
│  Subarray: [4, -1, 2]                │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 6: arr[6] = 1                  │
├──────────────────────────────────────┤
│  History: 5 (HELPFUL! ✅)             │
│  Choice A: Join → 5 + 1 = 6   ✅ WINNER │
│  Choice B: New  → 1                  │
│  bestEnding = 6                      │
│  result = max(5, 6) = 6       🎉 NEW BEST! │
│  Subarray: [4, -1, 2, 1]             │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 7: arr[7] = -5                 │
├──────────────────────────────────────┤
│  History: 6 (HELPFUL! ✅)             │
│  Choice A: Join → 6 + (-5) = 1  ✅ WINNER │
│  Choice B: New  → -5                 │
│  bestEnding = 1                      │
│  result = max(6, 1) = 6              │
│  Subarray: [4, -1, 2, 1, -5]         │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 8: arr[8] = 4                  │
├──────────────────────────────────────┤
│  History: 1 (HELPFUL! ✅)             │
│  Choice A: Join → 1 + 4 = 5   ✅ WINNER │
│  Choice B: New  → 4                  │
│  bestEnding = 5                      │
│  result = max(6, 5) = 6              │
│  Subarray: [4, -1, 2, 1, -5, 4]      │
└──────────────────────────────────────┘

✅ FINAL ANSWER: 6
🏆 WINNING SUBARRAY: [4, -1, 2, 1]
```

Color-Coded Decision Summary:

```
🎯 COLOR-CODED DECISION TRACE

Array: [-2,  1, -3,  4, -1,  2,  1, -5,  4]

Legend:
🔴 Start New (bestEnding = arr[i])
▯ Join Previous (bestEnding = bestEnding + arr[i])
⭐ New Global Maximum Found!

┌──────┬───────┬───────────────┬────────────┬─────────┬─────────┐
│Index │ Value │   Decision    │ bestEnding │ result  │ Subarray│
├──────┼───────┼───────────────┼────────────┼─────────┼─────────┤
│  0   │  -2   │ 🔴 Start (init) │    -2      │   -2    │  [-2]   │
├──────┼───────┼───────────────┼────────────┼─────────┼─────────┤
│  1   │   1   │ 🔴 Start New   │     1      │ ⭐ 1    │   [1]   │
│      │       │ (-2+1=-1 vs 1) │            │         │         │
├──────┼───────┼───────────────┼────────────┼─────────┼─────────┤
│  2   │  -3   │ ▯ Join Prev   │    -2      │    1    │ [1,-3]  │
│      │       │ (1-3=-2 vs -3) │            │         │         │
├──────┼───────┼───────────────┼────────────┼─────────┼─────────┤
│  3   │   4   │ 🔴 Start New   │     4      │ ⭐ 4    │   [4]   │
│      │       │ (-2+4=2 vs 4)  │            │         │         │
├──────┼───────┼───────────────┼────────────┼─────────┼─────────┤
│  4   │  -1   │ ▯ Join Prev   │     3      │    4    │ [4,-1]  │
│      │       │ (4-1=3 vs -1)  │            │         │         │
├──────┼───────┼───────────────┼────────────┼─────────┼─────────┤
│  5   │   2   │ ▯ Join Prev   │     5      │ ⭐ 5    │ [4,-1,2]│
│      │       │ (3+2=5 vs 2)   │            │         │         │
├──────┼───────┼───────────────┼────────────┼─────────┼─────────┤
│  6   │   1   │ ▯ Join Prev   │     6      │ ⭐ 6    │[4,-1,2,1]│
│      │       │ (5+1=6 vs 1)   │            │         │         │
├──────┼───────┼───────────────┼────────────┼─────────┼─────────┤
│  7   │  -5   │ ▯ Join Prev   │     1      │    6    │[4,-1,2, │
│      │       │ (6-5=1 vs -5)  │            │         │  1,-5]  │
├──────┼───────┼───────────────┼────────────┼─────────┼─────────┤
│  8   │   4   │ ▯ Join Prev   │     5      │    6    │[4,-1,2, │
│      │       │ (1+4=5 vs 4)   │            │         │  1,-5,4]│
└──────┴───────┴───────────────┴────────────┴─────────┴─────────┘

PATTERN OBSERVED:
🔴▯▯🔴▯▯▯▯▯
Start → Join → Join → Start → Join → Join → Join → Join → Join

FINAL ANSWER: 6 from subarray [4, -1, 2, 1]
```

Visual Graph Timeline:


📈 VISUAL GRAPH: bestEnding & result Over Time

Key Observations:
- bestEnding goes UP and DOWN (local decisions)
- result only goes UP (keeps global max)
- Peaks in bestEnding = potential new maximums

Scenario Comparisons:

```
😎 VISUAL COMPARISON: Impact of Elements

Scenario A: All Positive Numbers
────────────────────────────────────────
Array: [1, 2, 3, 4, 5]

  ●──●──●──●──●   (result always increasing)
  1   3   6   10  15

Decision at every step: ▯ JOIN (always helpful!)
Pattern: Linear growth ↗

Scenario B: Mixed Positive and Negative
────────────────────────────────────────
Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

     ●          ●──●──●──●            (result: -2→1→4→6)
    / \        /
   ●   ●   ●

Decisions: 🔴 🔴 ▯ 🔴 ▯ ▯ ▯ ▯ ▯
Pattern: Restart when burden, join when helpful

Scenario C: All Negative Numbers
────────────────────────────────────────
Array: [-5, -2, -8, -1, -3]

          ●  (result = -1, the least negative)
         /
        /
      ●
     /
   ●

Decision: Always 🔴 START NEW (pick least negative)
Pattern: Find maximum (closest to zero)
```

Before vs After Understanding:

```
🧠 BEFORE vs AFTER UNDERSTANDING KADANE'S

BEFORE (Confused 😵):
────────────────────────────────────────
Problem: Find max subarray sum in [-2, 1, -3, 4, -1, 2, 1]

Thinking: "I need to check ALL possible subarrays!"
          "That's O(n²) or O(n³)... too slow!"

Attempts:
  [-2] = -2
  [-2, 1] = -1
  [-2, 1, -3] = -4
  [1] = 1
  [1, -3] = -2
  [1, -3, 4] = 2
  ...

Total: 28 subarrays to check! 😩

AFTER (Enlightened 💡):
────────────────────────────────────────
Problem: Find max subarray sum in [-2, 1, -3, 4, -1, 2, 1]

Thinking: "At each step, I make ONE decision:"
          "Join the party OR start new!"

Process:
  i=0: -2 → start (no choice)
  i=1:  1 → start new (-2 is burden)
  i=2: -3 → join (1 is helpful)
  i=3:  4 → start new (-2 is burden)
  i=4: -1 → join (4 is helpful)
  i=5:  2 → join (3 is helpful)
  i=6:  1 → join (5 is helpful)

Result: 6 in O(n) time! ⚡

THE INSIGHT:
┌──────────────────────────────────────┐
│ Don't check all subarrays!           │
│ Instead: Make smart LOCAL decisions  │
│ that lead to GLOBAL optimum!         │
└──────────────────────────────────────┘
```

>

Java Code:

```java
public int maxSubArray(int[] nums) {
    // Handle empty array
    if (nums.length == 0) return 0;

    // Initialize with first element
    int bestEnding = nums[0];  // Best sum ending at current index
    int result = nums[0];      // Global maximum

    // Iterate through array starting from index 1
    for (int i = 1; i < nums.length; i++) {
        // THE DECISION: Join previous party OR start new party
        // Option A: Extend previous subarray (bestEnding + nums[i])
        // Option B: Start fresh from current element (nums[i])
        bestEnding = Math.max(bestEnding + nums[i], nums[i]);

        // Update global result if we found a better sum
        result = Math.max(result, bestEnding);
    }

    return result;
}

// Time Complexity: O(n) - single pass through array
// Space Complexity: O(1) - only use two variables
```

💡 Key Insights:

- ✓ At each index, compare: (previous + current) vs (current alone)
- ✓ If previous sum is negative, it's a burden - start fresh
- ✓ If previous sum is positive, it's helpful - extend it
- ✓ Always track global maximum separately

6. Problem 2: Minimum Subarray Sum 📉

Problem Statement:

Given an integer array, find the contiguous subarray which has the **smallest (minimum)** sum.

Example:

```
Input: nums = [3, -4, 2, -3, -1, 7, -5]
Output: -6
Explanation: [-4, 2, -3, -1] has the smallest sum = -6
```

Key Insight:

Exact same logic as Maximum Sum, but flip the comparison!

- Instead of choosing `max(prev+curr, curr)`, we choose **min(prev+curr, curr)**
- Instead of tracking maximum result, we track **minimum result**
- We want the MOST NEGATIVE sum (or closest to negative infinity)

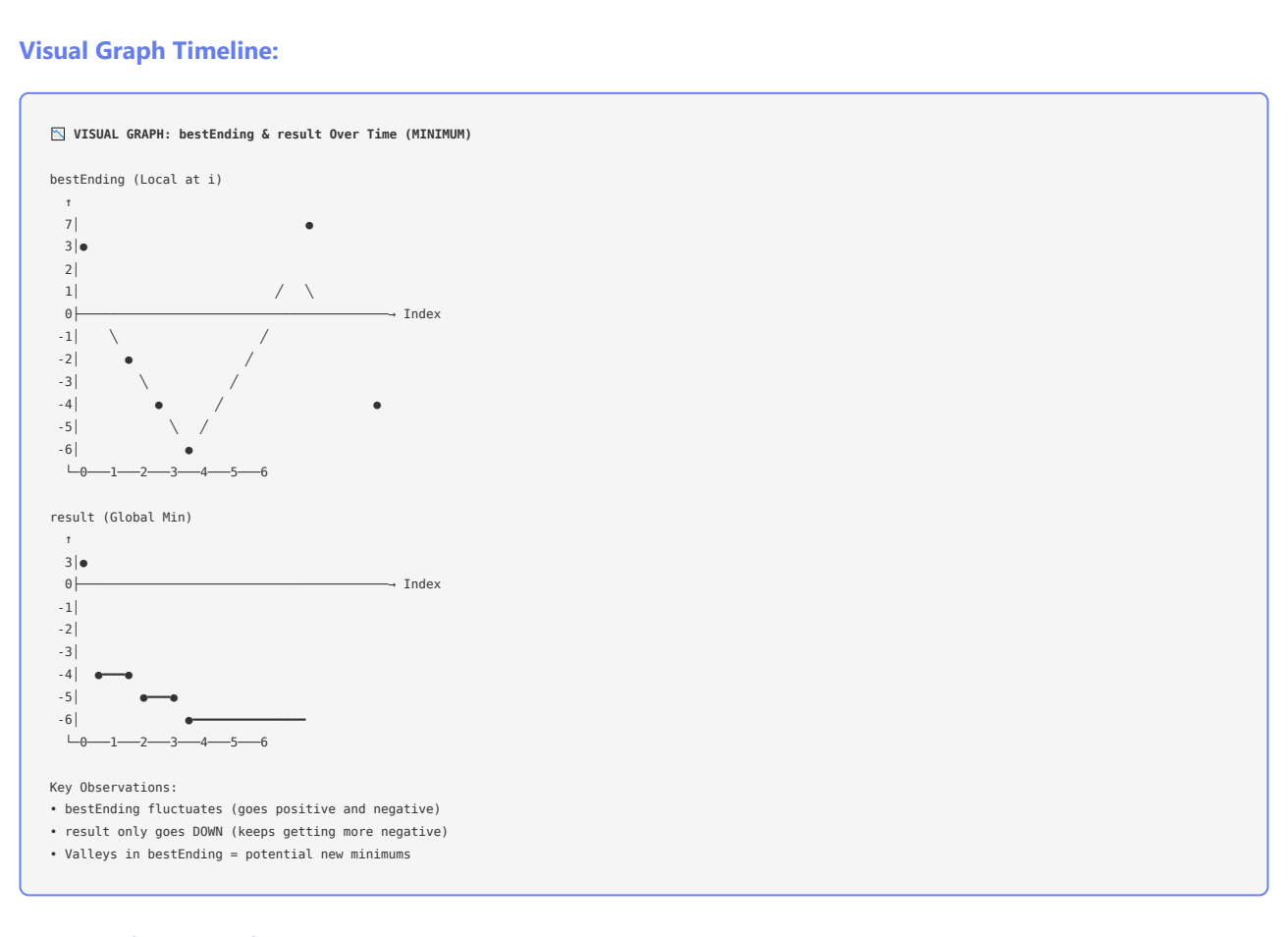
Detailed Step-by-Step Walkthrough:

```
🔍 DETAILED TRACE: Minimum Subarray Sum

Array: [3, -4, 2, -3, -1, 7, -5]

┌──────────────────────────────────────┐
│ INDEX 0: arr[0] = 3                  │
├──────────────────────────────────────┤
│  No history (first element)          │
│  bestEnding = 3                      │
│  result = 3                          │
│  Subarray: [3]                       │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 1: arr[1] = -4                 │
├──────────────────────────────────────┤
│  History: 3 (BURDEN for min! ❌)      │
│  Choice A: Join → 3 + (-4) = -1      │
│  Choice B: New  → -4      ✅ WINNER (more negative) │
│  bestEnding = -4                     │
│  result = min(3, -4) = -4            │
│  Subarray: [-4]                      │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 2: arr[2] = 2                  │
├──────────────────────────────────────┤
│  History: -4 (HELPFUL for min! ✅)    │
│  Choice A: Join → -4 + 2 = -2  ✅ WINNER │
│  Choice B: New  → 2          (too positive) │
│  bestEnding = -2                     │
│  result = min(-4, -2) = -4           │
│  Subarray: [-4, 2]                   │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 3: arr[3] = -3                 │
├──────────────────────────────────────┤
│  History: -2 (HELPFUL for min! ✅)    │
│  Choice A: Join → -2 + (-3) = -5  ✅ WINNER │
│  Choice B: New  → -3         (less negative) │
│  bestEnding = -5                     │
│  result = min(-4, -5) = -5           │
│  Subarray: [-4, 2, -3]               │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 4: arr[4] = -1                 │
├──────────────────────────────────────┤
│  History: -5 (HELPFUL for min! ✅)    │
│  Choice A: Join → -5 + (-1) = -6  ✅ WINNER │
│  Choice B: New  → -1         (less negative) │
│  bestEnding = -6                     │
│  result = min(-5, -6) = -6     🎉 NEW MINIMUM! │
│  Subarray: [-4, 2, -3, -1]           │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 5: arr[5] = 7                  │
├──────────────────────────────────────┤
│  History: -6 (HELPFUL for min! ✅)    │
│  Choice A: Join → -6 + 7 = 1   ✅ WINNER │
│  Choice B: New  → 7          (too positive) │
│  bestEnding = 1                      │
│  result = min(-6, 1) = -6            │
│  Subarray: [-4, 2, -3, -1, 7]        │
└──────────────────────────────────────┘

┌──────────────────────────────────────┐
│ INDEX 6: arr[6] = -5                 │
├──────────────────────────────────────┤
│  History: 1 (BURDEN for min! ❌)      │
│  Choice A: Join → 1 + (-5) = -4      │
│  Choice B: New  → -5        ✅ WINNER │
│  bestEnding = -5                     │
│  result = min(-6, -5) = -6           │
│  Subarray: [-5]                      │
└──────────────────────────────────────┘

✅ FINAL ANSWER: -6
🏆 WINNING SUBARRAY: [-4, 2, -3, -1]
```

Color-Coded Decision Summary:

```
🎯 COLOR-CODED DECISION TRACE (MINIMUM)

Array: [3, -4, 2, -3, -1, 7, -5]

Legend:
🔴 Start New (bestEnding = arr[i])
▯ Join Previous (bestEnding = bestEnding + arr[i])
⭐ New Global Minimum Found!

┌──────┬───────┬───────────────┬────────────┬─────────┬──────────┐
│Index │ Value │   Decision    │ bestEnding │ result  │ Subarray │
├──────┼───────┼───────────────┼────────────┼─────────┼──────────┤
│  0   │   3   │ 🔴 Start (init) │     3      │    3    │   [3]    │
├──────┼───────┼───────────────┼────────────┼─────────┼──────────┤
│  1   │  -4   │ 🔴 Start New   │    -4      │ ⭐ -4   │  [-4]    │
│      │       │ (3-4=-1 vs -4) │            │         │          │
├──────┼───────┼───────────────┼────────────┼─────────┼──────────┤
│  2   │   2   │ ▯ Join Prev   │    -2      │   -4    │ [-4, 2]  │
│      │       │ (-4+2=-2 vs 2) │            │         │          │
├──────┼───────┼───────────────┼────────────┼─────────┼──────────┤
│  3   │  -3   │ ▯ Join Prev   │    -5      │ ⭐ -5   │[-4,2,-3] │
│      │       │ (-2-3=-5 vs -3)│            │         │          │
├──────┼───────┼───────────────┼────────────┼─────────┼──────────┤
│  4   │  -1   │ ▯ Join Prev   │    -6      │ ⭐ -6   │[-4,2,-3, │
│      │       │ (-5-1=-6 vs -1)│            │         │   -1]    │
├──────┼───────┼───────────────┼────────────┼─────────┼──────────┤
│  5   │   7   │ ▯ Join Prev   │     1      │   -6    │[-4,2,-3, │
│      │       │ (-6+7=1 vs 7)  │            │         │  -1, 7]  │
├──────┼───────┼───────────────┼────────────┼─────────┼──────────┤
│  6   │  -5   │ 🔴 Start New   │    -5      │   -6    │  [-5]    │
│      │       │ (1-5=-4 vs -5) │            │         │          │
└──────┴───────┴───────────────┴────────────┴─────────┴──────────┘

PATTERN OBSERVED:
🔴🔴▯▯▯▯🔴
Start → Start → Join → Join → Join → Join → Start

FINAL ANSWER: -6 from subarray [-4, 2, -3, -1]

KEY DIFFERENCE FROM MAX SUM:
• For MAX: Negative history is BURDEN, positive history is HELPFUL
• For MIN: POSITIVE history is BURDEN, negative history is HELPFUL
```

Visual Graph Timeline:


📉 VISUAL GRAPH: bestEnding & result Over Time (MINIMUM)

Key Observations:
- bestEnding fluctuates (goes positive and negative)
- result only goes DOWN (keeps getting more negative)
- Valleys in bestEnding = potential new minimums

Max vs Min Comparison:

```
😎 MAX SUM vs MIN SUM: Decision Flip

SAME ARRAY: [3, -4, 2, -3, -1, 7, -5]

┌──────────────────────────────────────────┐
│          MAXIMUM SUBARRAY SUM            │
├──────────────────────────────────────────┤
│  Decision: Choose MAX(prev+curr, curr)   │
│                                          │
│  Index 5: arr[5] = 7                     │
│  • History: Something negative/small     │
│  • Choice: Start NEW at 7 ✅              │
│  • Why: 7 alone is better than negative + 7 │
│                                          │
│  RESULT: Finds MAXIMUM sum = 7 (just [7]) │
└──────────────────────────────────────────┘

┌──────────────────────────────────────────┐
│          MINIMUM SUBARRAY SUM            │
├──────────────────────────────────────────┤
│  Decision: Choose MIN(prev+curr, curr)   │
│                                          │
│  Index 5: arr[5] = 7                     │
│  • History: -6 (very negative! helpful!) │
│  • Choice: JOIN previous -6+7=1 ✅        │
│  • Why: Even though result is positive, it's less │
│         positive than 7 alone            │
│                                          │
│  RESULT: Finds MINIMUM sum = -6 ([-4,2,-3,-1]) │
└──────────────────────────────────────────┘

THE FLIP:
────────────────────────────────────────
MAX: Negative history = BURDEN → Start new
MIN: POSITIVE history = BURDEN → Start new

MAX: Positive history = HELPFUL → Join
MIN: NEGATIVE history = HELPFUL → Join
```

Java Code:

```java
public int minSubArray(int[] nums) {
    // Handle empty array
    if (nums.length == 0) return 0;

    // Initialize with first element
    int bestEnding = nums[0];  // Min sum ending at current index
    int result = nums[0];      // Global minimum

    // Iterate through array starting from index 1
    for (int i = 1; i < nums.length; i++) {
        // THE DECISION: Join previous party OR start new party
        // KEY DIFFERENCE: Use Math.MIN instead of Math.MAX!
        // Option A: Extend previous subarray (bestEnding + nums[i])
        // Option B: Start fresh from current element (nums[i])
        bestEnding = Math.min(bestEnding + nums[i], nums[i]);

        // Update global result if we found a smaller sum
        result = Math.min(result, bestEnding);
    }

    return result;
}

// Time Complexity: O(n) - single pass through array
// Space Complexity: O(1) - only use two variables
```

💡 Key Insights for Minimum:

- ✓ Everything FLIPS compared to maximum sum
- ✓ NEGATIVE history is now HELPFUL (makes sum more negative)
- ✓ POSITIVE history is now BURDEN (makes sum less negative)
- ✓ Use `min()` instead of `max()` everywhere
- ✓ Looking for most negative (or closest to negative infinity)
- ✓ Same O(n) time, O(1) space efficiency

🔄 Comparison Summary:

Aspect	Maximum Sum	Minimum Sum
Decision	`max(prev+curr, curr)`	`min(prev+curr, curr)`
Update Result	`max(result, bestEnding)`	`min(result, bestEnding)`
Helpful History	Positive values	Negative values
Burden History	Negative values	Positive values
Goal	Largest sum (positive infinity)	Smallest sum (negative infinity)

7. Problem 3: Maximum Product Subarray 🔢

Problem Statement:

LeetCode 152 - Maximum Product Subarray

Given an integer array, find the contiguous subarray with the largest **product**.

```
Input: nums = [2, 3, -2, 4]
Output: 6
Explanation: [2, 3] has the largest product = 6
```

The Twist: Why We Need TWO Variables 🎭

Negative × Negative = Positive!

A negative number wants to be multiplied by the **minimum** (most negative) product to become a **huge positive**!

Detailed Visual Example:

```
🔢 MAXIMUM PRODUCT: Why Track Both Max & Min

The Magic of Negatives:
────────────────────────────────────────
Array: [2, 3, -2, 4]
Index:  0  1   2  3

State at Index 1:
┌───────────────────────┐
│ maxEnding = 6  (2 × 3) │
│ minEnding = 2          │
│ result = 6             │
└───────────────────────┘

Now at Index 2: arr[2] = -2 (NEGATIVE!)
────────────────────────────────────────

Three Choices:

┌───────────────────────────────┐
│ Choice 1: Start New           │
│   → -2                        │
│   Value: -2                   │
└───────────────────────────────┘

┌───────────────────────────────┐
│ Choice 2: Join Max (Positive × Neg) │
│   → 6 × (-2)                  │
│   Value: -12  😞 BAD           │
└───────────────────────────────┘

┌───────────────────────────────┐
│ Choice 3: Join Min (Positive × Neg) │
│   → 2 × (-2)                  │
│   Value: -4   😞 Also bad      │
└───────────────────────────────┘

Wait! What if minEnding was NEGATIVE?
────────────────────────────────────────

Example: arr = [2, -5, -2]
At Index 1: minEnding = -10 (2 × -5)

Now at Index 2: arr[2] = -2

┌───────────────────────────────┐
│ Choice 3: Join Min (Neg × Neg) │
│   → (-10) × (-2)              │
│   Value: +20  🎉 HUGE POSITIVE! │
└───────────────────────────────┘

Key Insight:
┌──────────────────────────────────────────┐
│ Negative current number wants:           │
│ • minEnding (most negative) to flip into BIG + │
│                                          │
│ Positive current number wants:           │
│ • maxEnding (biggest positive) for bigger product │
└──────────────────────────────────────────┘
```

Simple Visual Example:

```
Current Element: -4
────────────────────────────────────────
Choice 1: Start New        → -4
Choice 2: Join Max (3)     → -4 × 3 = -12
Choice 3: Join Min (-6)    → -4 × (-6) = +24  ← WINNER!

Strategy: Track BOTH maxEnding and minEnding
```

Java Code:

```java
public int maxProduct(int[] nums) {
    if (nums.length == 0) return 0;

    // Must track BOTH max and min (negatives flip signs!)
    int maxEnding = nums[0];
    int minEnding = nums[0];
    int result = nums[0];

    for (int i = 1; i < nums.length; i++) {
        // Calculate three choices:
        // 1. Current element alone
        // 2. Current × Previous Max
        // 3. Current × Previous Min (important for negatives!)

        int choice1 = nums[i];
        int choice2 = maxEnding * nums[i];
        int choice3 = minEnding * nums[i];

        // Update current max and min
        maxEnding = Math.max(choice1, Math.max(choice2, choice3));
        minEnding = Math.min(choice1, Math.min(choice2, choice3));

        result = Math.max(result, maxEnding);
    }

    return result;
}
```

💡 Why Track Both Max and Min?

- **Positive current** → wants maxEnding (big positive × positive = bigger positive)
- **Negative current** → wants minEnding (big negative × negative = big positive)
- **Example:** -2 × (-10) = 20 (good!), but -2 × 5 = -10 (bad!)

11. Quick Reference Summary 📚

Kadane's Algorithm Decision Tree:

Problem Type	Key Decision	Variables Needed
Max Sum	max(prev + curr, curr)	bestEnding, result
Min Sum	min(prev + curr, curr)	bestEnding, result
Max Product	Track both max and min	maxEnding, minEnding, result
One Deletion	Track with/without deletion	noDelete, oneDelete, result
Absolute Sum	Run both max and min	maxSum, minSum
Circular	max(normal, total-min)	maxSum, minSum, totalSum

✅ Master Checklist

- ☐ Understand the "join vs start new" decision
- ☐ Can identify when to use Kadane's vs Sliding Window
- ☐ Know how to handle negative numbers
- ☐ Understand why tracking min/max both is needed for products
- ☐ Can handle one deletion variation
- ☐ Understand circular array logic (total - min)
- ☐ Can write code without memorization

🎓 You've Mastered Kadane's Algorithm!

You now understand:
✓ The core "join vs start new" decision pattern
✓ Why it works when sliding window fails
✓ How to adapt it for different problems
✓ The thought process behind each decision

Ready to solve any subarray problem! 🚀

TER_SLIDING_WINDOW_TABLE